



Instituto Tecnológico de Mazatlán



**Comparación entre delay() y millis() para el Control de Múltiples LEDs
en Arduino UNO R4 WiFi**

Carrera:

Ingeniería en Sistemas Computacionales

Materia:

Sistemas Programables

Profesor:

Miguel Hernandez Barrón

Alumnos:

Rosa Guadalupe Romero Navarro

Javier Ramón Quintero Valdez

Gabriel Alejandro Barrón Sarabia

1. Introducción

En el desarrollo de sistemas embebidos, la forma en que se maneja el tiempo dentro del programa es determinante para el comportamiento general del sistema. La función `delay()` es una de las primeras herramientas que se aprenden en Arduino para generar pausas, pero su uso indiscriminado bloquea por completo la ejecución del microcontrolador, impidiendo atender otras tareas mientras transcurre la espera.

Como alternativa, la función `millis()` permite implementar temporización no bloqueante: en lugar de detener el programa, se compara continuamente el tiempo transcurrido desde una referencia con el intervalo deseado, dejando al procesador libre para ejecutar otras tareas en paralelo.

En esta práctica se desarrollaron dos versiones de un mismo programa —control de tres LEDs con distintos intervalos de parpadeo (500 ms, 1000 ms y 1500 ms)— utilizando primero `delay()` y después `millis()`, con el fin de comparar de forma práctica las diferencias entre ambos enfoques. Además, en la versión con `millis()` se incorporó una tarea adicional (impresión periódica por el puerto serie) para evidenciar la capacidad de ejecutar múltiples tareas de forma concurrente.

2. Objetivos

Objetivo general:

- Comparar de forma práctica el uso de `delay()` frente a `millis()` para el control de temporización de múltiples LEDs en Arduino UNO R4 WiFi.

Objetivos específicos:

- Implementar el control de tres LEDs con intervalos distintos utilizando únicamente `delay()`, identificando sus limitaciones.
- Reimplementar el mismo control utilizando `millis()`, logrando que cada LED parpadee de forma independiente y simultánea.
- Agregar una tarea adicional (mensaje por puerto serie cada 3000 ms) para comprobar que la temporización no bloqueante permite ejecutar múltiples procesos en paralelo.
- Analizar y documentar las diferencias observadas entre ambos enfoques.

3. Herramientas y material utilizado

- Arduino UNO R4 WiFi
- Arduino IDE
- Protoboard
- LEDs: rojo, amarillo y verde
- Resistencias limitadoras de corriente

- Cables de conexión (jumpers)

4. Desarrollo

4.1 Diagrama del circuito

El circuito consiste en tres LEDs (rojo, amarillo y verde) conectados a las salidas digitales del Arduino a través de resistencias limitadoras de corriente. El diagrama esquemático, elaborado en Tinkercad, se muestra en la Figura 1.

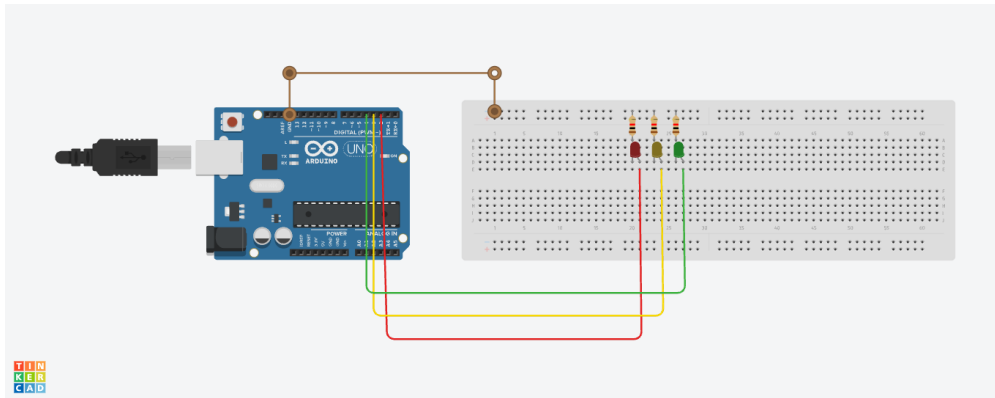


Figura 1. Diagrama esquemático del circuito (Tinkercad): Arduino UNO y tres LEDs con resistencias.

4.2 Parte 1 — Antipatrón con delay()

En esta primera versión, cada LED se enciende y apaga utilizando delay() de forma secuencial. Aunque el código es sencillo de leer, el tiempo total de cada ciclo de loop() es la suma de todos los delay() (6000 ms en total), por lo que ningún LED respeta realmente su intervalo nominal de forma independiente.

```
const int LED1 = 2;
const int LED2 = 3;
const int LED3 = 4;

void setup() {
  pinMode(LED1, OUTPUT);
  pinMode(LED2, OUTPUT);
  pinMode(LED3, OUTPUT);
}

void loop() {
  digitalWrite(LED1, HIGH); delay(500); digitalWrite(LED1, LOW); delay(500);
  digitalWrite(LED2, HIGH); delay(1000); digitalWrite(LED2, LOW); delay(1000);
  digitalWrite(LED3, HIGH); delay(1500); digitalWrite(LED3, LOW); delay(1500);
}
```

4.3 Parte 2 — Temporización no bloqueante con millis()

En la segunda versión, cada LED tiene su propia marca de tiempo (previoLEDx) y su propio intervalo. En cada vuelta de loop() se compara el tiempo transcurrido contra el intervalo correspondiente y, si se cumple, se invierte el estado del LED. Se añadió además una cuarta tarea que imprime un mensaje por el puerto serie cada 3000 ms, totalmente independiente de las demás.

```
unsigned long previoLED1 = 0, previoLED2 = 0, previoLED3 = 0, previoSerial = 0;
const unsigned long INTERVALO_LED1 = 500;
const unsigned long INTERVALO_LED2 = 1000;
const unsigned long INTERVALO_LED3 = 1500;
const unsigned long INTERVALO_SERIAL = 3000;

void loop() {
  unsigned long ahora = millis();
  if (ahora - previoLED1 >= INTERVALO_LED1) { previoLED1 = ahora; estadoLED1 = !estadoLED1;
digitalWrite(LED1, estadoLED1); }
  if (ahora - previoLED2 >= INTERVALO_LED2) { previoLED2 = ahora; estadoLED2 = !estadoLED2;
digitalWrite(LED2, estadoLED2); }
  if (ahora - previoLED3 >= INTERVALO_LED3) { previoLED3 = ahora; estadoLED3 = !estadoLED3;
digitalWrite(LED3, estadoLED3); }
  if (ahora - previoSerial >= INTERVALO_SERIAL) { previoSerial = ahora; Serial.println("Hola
Mundo"); }
}
```

5. Evidencia del circuito funcionando

A continuación se presenta la fotografía del circuito físico armado en protoboard, conectado a la placa Arduino UNO R4 WiFi durante las pruebas de ambas versiones del programa.

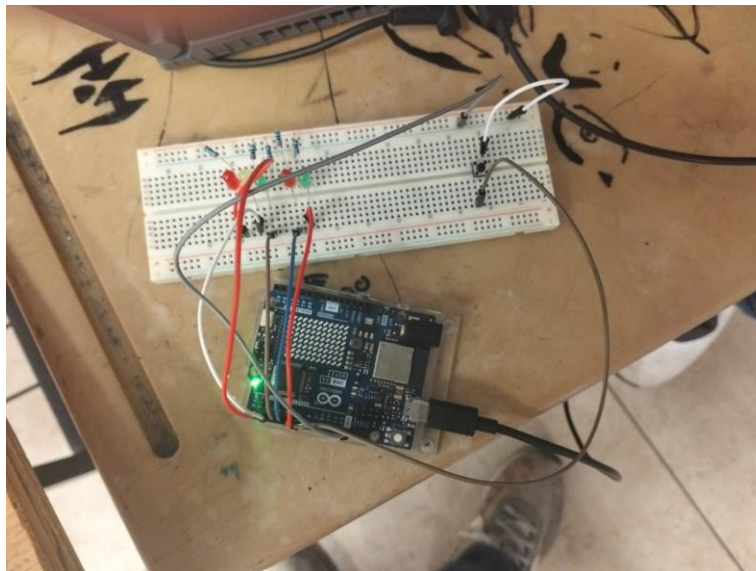


Figura 2. Circuito armado en protoboard con Arduino UNO R4 WiFi.

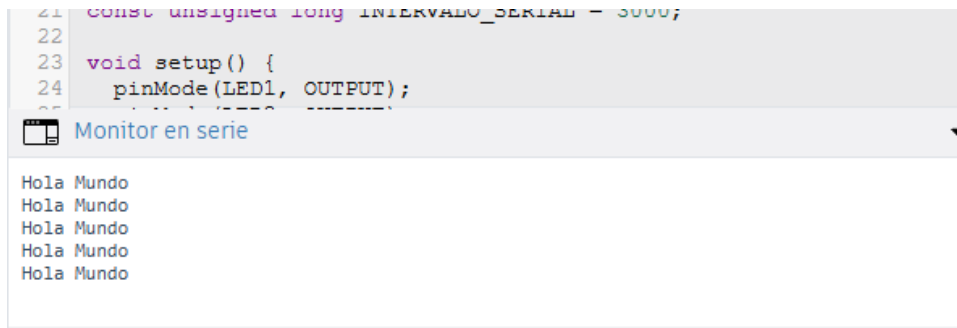


Figura 3. Captura del monitor en serie mostrando el mensaje "Hola Mundo" impreso cada 3000 ms por la tarea adicional del código con millis().

6. Resultados: comparación delay() vs millis()

Durante las pruebas se observó un comportamiento claramente distinto entre ambas versiones del programa. La siguiente tabla resume la comparación:

Aspecto	Con delay()	Con millis()
Ejecución del programa	Se bloquea por completo durante cada delay(); nada más puede ejecutarse mientras tanto.	El programa nunca se detiene; cada tarea se evalúa en cada vuelta del loop().
Independencia de tareas	Los LEDs no parpadean de forma independiente: los tiempos se suman en secuencia (0.5 + 1 + 1.5 s por ciclo).	Cada LED mantiene su propio intervalo (500, 1000, 1500 ms) de forma simultánea e independiente.
Tarea extra (mensaje serial)	No es posible agregar una tarea periódica adicional sin afectar los tiempos existentes.	La tarea del monitor serie (cada 3000 ms) corre en paralelo sin alterar el parpadeo de los LEDs.
Escala bilidad	Agregar más LEDs o sensores incrementa el tiempo total de bloqueo del ciclo.	Agregar más tareas solo implica una nueva variable de tiempo y una nueva condición en el loop().
Respuesta a eventos externos	Un botón u otro sensor no podría leerse mientras el programa está en delay().	Los eventos externos pueden revisarse en cada iteración, sin demoras.

En la versión con delay(), los LEDs se encendían y apagaban en secuencia estricta, dando la impresión de un parpadeo dependiente entre sí y con un ciclo total mucho más largo (6 segundos) que la suma de sus intervalos nominales. No fue posible agregar la tarea del monitor serie sin alterar aún más los tiempos.

En la versión con millis(), los tres LEDs parpadearon de forma simultánea, cada uno respetando su propio intervalo (500, 1000 y 1500 ms), y el mensaje por el puerto serie se imprimió cada 3 segundos sin afectar el comportamiento de los LEDs, confirmando la capacidad de esta técnica para ejecutar múltiples tareas de forma concurrente dentro de un mismo loop().

7. Conclusiones individuales

Javier

Esta práctica dejó muy clara la diferencia entre bloquear el procesador con `delay()` y liberar su tiempo con `millis()`. Ver en la misma sesión ambas versiones del código, una al lado de la otra, permitió entender de forma concreta por qué `millis()` es indispensable cuando un sistema necesita atender varias tareas o eventos externos al mismo tiempo, como ocurrirá más adelante con el semáforo y su botón peatonal.

Gabriel

Comparar ambas versiones me ayudó a notar que el problema de `delay()` no es solo la pausa en sí, sino que esa pausa se acumula: al sumar los tres `delay()` de cada LED, el ciclo completo termina siendo mucho más largo de lo que uno esperaría al ver el código. Con `millis()`, en cambio, cada tarea vive de forma independiente gracias a su propia variable de tiempo, lo que hace que el programa escale mucho mejor cuando se agregan más LEDs o, como en este caso, una tarea extra de impresión por puerto serie.

Rosa

Lo que más resalto de esta práctica es cómo un cambio relativamente pequeño en la forma de programar, pasar de `delay()` a comparaciones con `millis()`, transforma por completo el comportamiento del sistema sin necesidad de cambiar el hardware. Ver funcionando al mismo tiempo el parpadeo de los tres LEDs y el mensaje periódico por el monitor serie fue la mejor evidencia de que la temporización no bloqueante realmente permite que varias tareas convivan dentro de un mismo programa.